ASSIGNAMENT 5

First Aproach

My initial goal was to develop a fine-tuned license plate recognition system for `Harinera LaMeta`. This would have allowed us to avoid relying on Hikvision cameras and to improve performance in scenarios where they fail.

This approach turned out to be a dead end. Although it initially seemed viable, the complexity increased dramatically during the image-loading stage. Managing the data pipeline alone became impractical.

The abandoned approach is documented in `historico-transito-vehiculos.ipynb`. That notebook includes a method to download roughly 30 GB of correctly labeled license-plate images, split into training and test datasets.

Second Aproach

The second approach is to use an LLM to control specific parts of the factory. This will be implemented using:

- MCP
- LM Studio

The subsystem that is most ready for LLM integration is the one that controls truck flow, internally known as `Transito`.

The goal is to allow an operator to easily grant temporary authorization to a vehicle, which in turn enables the gates to open automatically.

Since the implementation involves proprietary code that cannot be shared, I will provide a `mcp.patch` file containing the required changes. A video demonstrating the system in operation will also be included.

Implementation details

In practice, this is straightforward: the system only needs to insert a new entry into the `VehiculosAutorizados` SQL table. Once this record exists, the next time a camera detects that license plate, the vehicle will be authorized automatically.

Setting up the MCP

Since all the code at `Harinera LaMeta` must be .NET-based, we will extend the existing system rather than introduce a separate service.

The article at https://devblogs.microsoft.com/dotnet/build-a-model-context-protocol-mcp-server-in-csharp/ provides a solid starting point for implementing MCP in C#.

Because the system already uses `ASP.NET Core`, the required changes are minimal: we add the necessary dependency, define an MCP Tool, register it with the MCP engine, and expose that engine over HTTP by declaring the appropriate endpoint.

```
<PackageReference Include="ModelContextProtocol.AspNetCore" />
```

```
using System;
using System.ComponentModel;
using System.Threading.Tasks;
using Dapper;
using META.Web.LectorMatriculas.Models;
using Microsoft.Data.SqlClient;
using Microsoft.Extensions.Configuration;
using ModelContextProtocol.Server;

namespace META.Web.LectorMatriculas.Tools;

[McpServerToolType]
public class TransitoTool(IConfiguration configuration)
{

    private readonly IConfiguration configuration = configuration;

    [Description("Authorizes a Vehicle")]
    [McpServerTool]
    public async Task AutorizeAsync([Description("The Plate of the Vehicle to Autorize")] string plate)
    {
        await using var sqlConnection = new SqlConnection(this.configuration.GetConnectionString("TransitoVehiculos"));

        var vehiculoAutorizado = new VehiculosAutorizados
        {
            Conductor = string.Empty,
            DNI = string.Empty,
            FechaInicioVigencia = DateTime.Now,
            FechaFinVigencia = DateTime.Now + TimeSpan.FromDays(1),
            Matricula = plate,
            VehiculoInterno = false
        };
        await sqlConnection.ExecuteAsync($@"
            INSERT
                [VehiculosAutorizados](
                    {nameof(VehiculosAutorizados.Conductor)},
                    {nameof(VehiculosAutorizados.DNI)},
                    {nameof(VehiculosAutorizados.FechaFinVigencia)},
                    {nameof(VehiculosAutorizados.FechaInicioVigencia)},
                    {nameof(VehiculosAutorizados.Matricula)},
                    {nameof(VehiculosAutorizados.VehiculoInterno)}
                )
            VALUES
                (
                    @{nameof(VehiculosAutorizados.Conductor)},
                    @{nameof(VehiculosAutorizados.DNI)},
                    @{nameof(VehiculosAutorizados.FechaFinVigencia)},
                    @{nameof(VehiculosAutorizados.FechaInicioVigencia)},
                    @{nameof(VehiculosAutorizados.Matricula)},
                    @{nameof(VehiculosAutorizados.VehiculoInterno)}
                );
        ", vehiculoAutorizado);
    }

}
```

```
serviceCollection.AddMcpServer()
    .WithHttpTransport()
    .WithTools<TransitoTool>();
```

```
endpointRouteBuilder.MapMcp("/mcp");
```

Setting up the LM Studio

In LM Studio, we only need to modify the `mcp.json` file as follows:

```
{
  "mcpServers": {
    "remote": {
      "url": "http://localhost:5600/mcp"
    }
  }
}
```

Video

The video can be seen in ./recording.mp4

Conclusions

MCP provides a straightforward way to integrate LLMs into real applications, including legacy systems. It enables new interaction models that were previously impractical.

A future extension is to enable voice-based interaction with the LLM. This would allow operators to perform quick, low-risk actions, such as opening or closing doors.

Higher-risk operations, such as starting engines, must always require explicit supervisor approval.